

Lab 6

Part A - List comprehensions

1. Create squares for numbers 1-20 using a normal loop, then a list comprehension.
2. Create a list containing only even numbers from 1-100.
3. Convert a list of names to stripped, title-cased names.
4. Given scores, create a list containing only passing scores.
5. Create labels such as 'PASS'/'FAIL' for every score using a conditional expression in a comprehension.
6. Rewrite three earlier loop-based transformations from Lessons 2-4 as comprehensions.

Part B - Dictionary and set comprehensions

1. Create a dictionary mapping numbers 1-10 to their squares.
2. Given a list of words, create a dictionary mapping each word to its length.
3. Given a list with duplicates, create a set comprehension containing lowercase normalized values.
4. Create a dictionary of only products whose price is below a chosen threshold.
5. Create a dictionary mapping student names to PASS/FAIL from a list of student dictionaries.

Part C - enumerate

1. Print a playlist with numbering starting at 1 using enumerate.
2. Given a list of tasks, print 'Task 1:', 'Task 2:' etc.
3. Find and print indexes of all values above a threshold.
4. Rewrite a `range(len(...))` loop using enumerate and explain why the new version is clearer.

Part D - zip and unpacking

1. Combine separate name and score lists using zip and print each pair.
2. Create a dictionary using dict(zip(keys, values)).
3. Combine three lists: product name, price and stock.
4. Investigate what happens when zipped lists have different lengths.
5. Use tuple unpacking directly in a for loop over zipped data.
6. Swap two variables without a temporary variable.

Part E - sorted and lambda

1. Sort a list of words by length using sorted(..., key=...).
2. Sort a list of student dictionaries by score ascending and descending.
3. Sort products by price using a lambda.
4. Sort people by last name when each item is a dictionary containing first_name and last_name.
5. Write a normal named function for a sort key, then replace it with lambda. Compare when each is clearer.

Part F - Applied challenge: Data cleanup

1. Start with a list of at least twelve messy dictionaries representing products: inconsistent name casing/spacing, category, price and stock.
2. Create a cleaned list where names/categories are normalized. Use comprehensions where readable.
3. Create a list of in-stock products.
4. Create a set of unique normalized categories.
5. Create a dictionary mapping product name to inventory value (price * stock).
6. Sort products by inventory value from highest to lowest.
7. Use enumerate to print a ranked report.
8. Use zip to combine at least one pair of separate derived lists in a meaningful way.

9. Write both a deliberately over-complicated comprehension and a clearer alternative. Explain why the clearer version wins.

Part G - Stretch challenges

1. Flatten a simple list of lists using a comprehension.
2. Create a multiplication table structure using a nested comprehension, then decide whether the result is readable enough.
3. Given names and scores, create only passing student dictionaries in one readable comprehension.
4. Use `any()` and `all()` to answer useful questions about a score list, after first solving them with loops.
5. Create five examples where Pythonic syntax reduces boilerplate without reducing clarity.